Diary

Goal

Track the cleanup of `legacy-version/` (especially removing the auth subsystem that currently crashes the dev UI) with enough detail to resume mid-stream and to review changes safely.

Step 1: Create MO-001 ticket workspace

Created a dedicated ticket workspace for analyzing and simplifying the legacy frontend. This isolates the "investigation + cleanup" work from the earlier Go port ticket (PMQ-1) and gives us a place to write a long-form assessment before touching code.

What I did

- Ran `docmgr ticket create-ticket` for `MO-001-CLEANUP-LEGACY`.
- Added a `design-doc` for a textbook-style assessment and this diary `reference` doc.

Why

- The UI crash stack traces into legacy auth (`getLoginUrl` → `useAuth` → `Home`), so we need a structured teardown plan before removing pieces.

What worked

- Ticket workspace + docs were created under `markdown-quizz/ttmp/2026/01/19/...`.

What didn't work

- N/A

What I learned

- Docmgr creates a default `design/` dir, but our actual design documents in this repo live under `design-doc/` (template-based).

What was tricky to build

- N/A

What warrants a second pair of eyes

- N/A

What should be done in the future

- Relate the key legacy frontend files to this diary/design-doc once the survey is complete.

Code review instructions

- Open `markdown-quizz/ttmp/2026/01/19/MO-001-CLEANUP-LEGACY--cleanup-legacy-version-frontend/index.md` to see the ticket workspace scaffold.

Technical details

- Ticket root: `markdown-quizz/ttmp/2026/01/19/MO-001-CLEANUP-LEGACY--cleanup-legacy-version-frontend/`

Step 2: Survey legacy frontend and pinpoint the crash

Did a quick structural survey of `legacy-version/` with the specific goal of understanding why the UI is crashing at startup and what "auth" actually means in this codebase. The immediate culprit is that the login URL is computed eagerly from missing Vite env vars, so simply rendering pages that call `useAuth()` blows up before any API calls happen.

This clarified that removing auth (or at least removing OAuth/redirect behavior) is the right next step: the Go backend is already operating in a "synthetic admin user" mode, and the legacy frontend's OAuth flow is tied to Manus-specific infrastructure that we don't want to depend on.

What I did

- Located the Vite entrypoint `markdown-quizz/legacy-version/client/src/main.tsx` and confirmed it imports `getLoginUrl()`.
- Traced the reported stack: `getLoginUrl` (`src/const.ts`) → `useAuth` (`src/_core/hooks/useAuth.ts`) → `Home` (`src/pages/Home.tsx`) and also other pages/layouts.
- Identified all UI call sites via `rg`:
  - `getLoginUrl` is used in `main.tsx`, `DashboardLayout.tsx`, and multiple pages.
  - `useAuth` is used in `DashboardLayout.tsx` and most pages.

Why

- We need a clean "no-auth" frontend path to keep local dev working (no proprietary OAuth portal required).

What worked

- Root cause is clear: `getLoginUrl()` reads `import.meta.env.VITE_OAUTH_PORTAL_URL` / `VITE_APP_ID` and then calls `new URL(...)`. When env vars are empty/missing, it throws during render.

What didn't work

- N/A (this step was investigation only).

What I learned

- `legacy-version/` is structured like a monorepo:
  - `client/` is a Vite React SPA (Tailwind + shadcn/ui + Radix).
  - `server/` is an Express+tRPC server (now largely replaced by the Go backend).
  - `shared/` provides a few shared constants.
  - Several pieces (OAuth + Manus runtime) are Manus-specific and should be treated as "proprietary dependencies" for cleanup.

What was tricky to build

- The crash happens even when you don't "use auth" explicitly: `useAuth()` has a default parameter `redirectPath = getLoginUrl()` so it executes immediately on hook invocation.

What warrants a second pair of eyes

- N/A

What should be done in the future

- When removing auth, ensure pages that previously gated on `isAuthenticated` still behave sensibly (e.g., admin pages still reachable in local dev).

Code review instructions

- Start with `markdown-quizz/legacy-version/client/src/const.ts` and `markdown-quizz/legacy-version/client/src/_core/hooks/useAuth.ts`.
- Grep call sites: `rg -n "getLoginUrl|useAuth\\(" markdown-quizz/legacy-version/client/src`.

Technical details

- Crash signature (reported by user):
  - `getLoginUrl@http://localhost:3001/src/const.ts:7:15`
  - `useAuth@http://localhost:3001/src/_core/hooks/useAuth.ts:6:61`
  - `Home@http://localhost:3001/src/pages/Home.tsx:26:67`

Step 3: Write a textbook-style legacy frontend assessment

Produced a detailed, "explain it like a maintainer" writeup of how the legacy SPA is built and how it is structured, then followed with a subsystem-by-subsystem cleanup assessment. This gives us a stable reference point so we can remove auth cleanly without accidentally breaking unrelated parts (tRPC client wiring, routing, quiz rendering).

The key takeaway captured in the doc: the SPA is broadly conventional (Vite+React+tRPC+React Query+Tailwind+shadcn/ui), but it is coupled to Manus/OAuth via a small set of files and Vite plugins. That coupling is exactly what we can remove now.

What I did

- Filled out `design-doc/01-legacy-frontend-structure-cleanup-assessment.md` with:
  - Part 1: structure/build/runtime/pages
  - Part 2: cleanup assessment and proposed removals
- Related core legacy frontend files to the design doc for easier review.

Why

- Cleanup work is safer when the "current state" is documented and reviewable.

What worked

- The doc clearly isolates the auth/OAuth subsystem (files + call sites) from the rest of the UI.

What didn't work

- N/A

What I learned

- The biggest blocker is not the API layer; it's the Manus-specific OAuth portal URL construction that runs eagerly during render.

What was tricky to build

- Keeping the document "textbook style" while still being actionable: I treated "Part 1 = explanation" and "Part 2 = assessment + plan" as separate layers.

What warrants a second pair of eyes

- N/A

What should be done in the future

- Add a follow-up ticket to decouple the frontend's `AppRouter` type from `legacy-version/server/` so the legacy Express server can be deleted.

Code review instructions

- Start at `markdown-quizz/ttmp/2026/01/19/MO-001-CLEANUP-LEGACY--cleanup-legacy-version-frontend/design-doc/01-legacy-frontend-structure-cleanup-assessment.md`.
- Skim Part 1 to understand architecture, then jump to Part 2.2 ("Auth/OAuth subsystem") for the removal plan.

Technical details

- Design doc path:
  - `markdown-quizz/ttmp/2026/01/19/MO-001-CLEANUP-LEGACY--cleanup-legacy-version-frontend/design-doc/01-legacy-frontend-structure-cleanup-assessment.md`

Step 4: Add explicit cleanup tasks for auth removal

Translated the assessment into a concrete task list focused on removing auth/OAuth behavior from the SPA. This makes it easy to track what "remove auth code" means in practice (URL construction, redirects, gates, and Manus login UI), and keeps validation steps visible.

What I did

- Added ticket tasks for auth removal and validation via `docmgr task add`.
- Removed the template placeholder task from `tasks.md` to avoid confusion.

Why

- The auth code is spread across bootstrap, hooks, layout, and pages; a task list helps ensure we remove it comprehensively.

What worked

- The ticket now has a focused TODO list that maps directly to the crash root cause and removal plan.

What didn't work

- N/A

What I learned

- Docmgr's default `tasks.md` template includes a placeholder checkbox; it's better to delete it once real tasks are added.

What was tricky to build

- N/A

What warrants a second pair of eyes

- N/A

What should be done in the future

- Add follow-up tasks (in a separate ticket) for deleting `legacy-version/server/` after moving the frontend's `AppRouter` type out of that folder.

Code review instructions

- Review the task list at `markdown-quizz/ttmp/2026/01/19/MO-001-CLEANUP-LEGACY--cleanup-legacy-version-frontend/tasks.md`.

Technical details

- Tasks were added with `docmgr task add --ticket MO-001-CLEANUP-LEGACY --text ...`.

Step 5: Remove legacy auth/OAuth code from the SPA

Removed the Manus OAuth portal login flow and the associated "auth" UI gates from `legacy-version/client/`. The goal was to make the SPA runnable in a "no-auth" world (which matches the current Go backend behavior) and to fix the crash that happened when `getLoginUrl()` attempted to build a URL from missing Vite env vars.

This change is intentionally blunt: instead of trying to preserve a half-working login experience, the UI now assumes no-auth mode and stops redirecting users to an external login portal.

What I did

- Deleted `markdown-quizz/legacy-version/client/src/const.ts` (OAuth login URL generator).
- Deleted `markdown-quizz/legacy-version/client/src/_core/hooks/useAuth.ts` (auth hook) and removed the now-empty `_core/` directory.
- Removed redirect-to-login behavior from `markdown-quizz/legacy-version/client/src/main.tsx`.

- Removed all “Sign In Required” gates and sign-in links from pages:
  - `markdown-quizz/legacy-version/client/src/pages/Home.tsx`
  - `markdown-quizz/legacy-version/client/src/pages/Admin.tsx`
  - `markdown-quizz/legacy-version/client/src/pages/DocumentEditor.tsx`
  - `markdown-quizz/legacy-version/client/src/pages/MySubmissions.tsx`
  - `markdown-quizz/legacy-version/client/src/pages/Analytics.tsx`
  - `markdown-quizz/legacy-version/client/src/pages/SubmissionReview.tsx`
  - `markdown-quizz/legacy-version/client/src/pages/DocumentView.tsx`
- Simplified `markdown-quizz/legacy-version/client/src/components/DashboardLayout.tsx` to always render and show a “No-auth mode” dropdown label.
- Deleted unused Manus login UI component `markdown-quizz/legacy-version/client/src/components/M`

Why

- The auth subsystem is proprietary integration glue (Manus OAuth portal) and is currently a hard blocker: it crashes local dev when env vars are missing and redirects away from local pages.
- The Go backend currently behaves as if a synthetic admin user exists, so a login flow is not required to develop core features.

What worked

- TypeScript check passes:
  - `pnpm -C markdown-quizz/legacy-version check`
- Production UI build succeeds:
  - `pnpm -C markdown-quizz/legacy-version build:ui`
- Grep confirms auth call sites are gone:
  - `rg -n "useAuth\\(|getLoginUrl" markdown-quizz/legacy-version/client/src` (no matches)

What didn’t work

- I attempted to start the Vite dev server to validate “renders in dev” and hit a bind permission error:
  - Command: `pnpm -C markdown-quizz/legacy-version dev:ui --host 127.0.0.1 --port 3001`
  - Error:

    ```
    * Error: listen EPERM: operation not permitted 127.0.0.1:3001
    ```

  - This appears to be an environment restriction rather than a code issue.

What I learned

- The most fragile part of the legacy auth system was not the tRPC procedures; it was the eager evaluation of OAuth URL construction during render.

What was tricky to build

- Removing auth safely required touching many pages because the “sign-in required” UX was duplicated (not centralized).

What warrants a second pair of eyes

- Ensure the “no-auth mode” assumption is acceptable for the intended deployment path (i.e., we aren’t accidentally deleting required real auth for prod without replacement).

What should be done in the future

- Create a follow-up ticket to remove `legacy-version/server/_core/oauth.ts` / Manus SDK integration and to relocate `AppRouter` types out of `legacy-version/server/` so the legacy backend can be deleted.
- Decide whether `vite-plugin-manus-runtime` should be removed as well (separate, but related).

Code review instructions

- Review the removed-auth diff by scanning:
  - `markdown-quizz/legacy-version/client/src/main.tsx`
  - `markdown-quizz/legacy-version/client/src/pages/*`
  - `markdown-quizz/legacy-version/client/src/components/DashboardLayout.tsx`
- Validate locally (outside this environment restriction) with:
  - `pnpm -C markdown-quizz/legacy-version dev:ui`
  - `pnpm -C markdown-quizz/legacy-version build:ui`

Technical details

- Removed files:
  - `markdown-quizz/legacy-version/client/src/const.ts`
  - `markdown-quizz/legacy-version/client/src/_core/hooks/useAuth.ts`
  - `markdown-quizz/legacy-version/client/src/components/ManusDialog.tsx`

Step 6: Ignore local SQLite dev artifacts

While validating the frontend changes, local SQLite files were generated in the `markdown-quizz/` repo root and showed up as untracked files. These are runtime artifacts and should never be committed, so I added explicit ignore rules.

What I did

- Updated `markdown-quizz/.gitignore` to ignore `*.sqlite`, `*.sqlite-wal`, and `*.sqlite-shm`.

Why

- Keeps the repo clean and prevents accidental commits of local database state.

What worked

- The ignore patterns match the observed untracked filenames (`markdown-quizz.sqlite*`).

What didn’t work

- N/A

What I learned

- The current default `.gitignore` didn't include database artifact patterns; it's worth keeping these in the project's ignore list since the Go server uses SQLite for local dev.

What was tricky to build

- N/A

What warrants a second pair of eyes

- N/A

What should be done in the future

- Consider standardizing the default DB path (e.g., always under a `var/` or `data/` directory) to make cleanup even simpler.

Code review instructions

- Review `markdown-quizz/.gitignore`.

Technical details

- Ignore entries added:
  - `*.sqlite`
  - `*.sqlite-shm`
  - `*.sqlite-wal`

Step 7: Fix doc vocabulary warnings

While running `docmgr doctor` on the new ticket, the repo-wide vocabulary file was empty, so the ticket index's `Topics`, `DocType`, `Intent`, and `Status` were flagged as "unknown". I added the minimal vocabulary entries needed so doctor checks pass for this ticket (and future ones).

What I did

- Added vocabulary entries in `markdown-quizz/ttmp/vocabulary.yaml` for:
  - topics: `frontend`, `legacy`, `cleanup`
  - docTypes: `index`, `design-doc`, `reference`
  - intent: `long-term`
  - status: `active`
- Re-ran `docmgr doctor --ticket MO-001-CLEANUP-LEGACY` to confirm clean.

Why

- Keeps ticket metadata consistent and makes docmgr validation useful rather than noisy.

What worked

- `docmgr doctor` now reports "All checks passed" for `MO-001-CLEANUP-LEGACY`.

What didn't work

- I initially tried `docmgr vocab add --category doctype ...` following the doctor hint, but the correct category name is `docTypes`.

What I learned

- The vocabulary categories in this repo are `topics`, `docTypes`, `intent`, and `status` (matching `markdown-quizz/ttmp/vocabulary.yaml`).

What was tricky to build

- N/A

What warrants a second pair of eyes

- N/A

What should be done in the future

- Consider seeding the repo vocabulary with the standard doc types from `markdown-quizz/ttmp/_templates/` so new tickets don't reintroduce warnings.

Code review instructions

- Review `markdown-quizz/ttmp/vocabulary.yaml`.

Technical details

- Validation command: `docmgr doctor --ticket MO-001-CLEANUP-LEGACY --stale-after 30`

Step 8: Validation status and keeping local sqlite files

Validated the auth removal at the level we can in this environment: TypeScript type-check passes and production builds succeed. Starting the Vite dev server is blocked by an environment-level EPERM bind error, so "renders in dev" needs to be confirmed on a machine/environment that allows binding the chosen port.

Also confirmed we should keep local SQLite files in the repo working tree (but ignore them in git), so local dev state can persist without polluting commits.

What I did

- Ran:
  - `pnpm -C markdown-quizz/legacy-version check`
  - `pnpm -C markdown-quizz/legacy-version build:ui`
- Attempted to run:
  - `pnpm -C markdown-quizz/legacy-version dev:ui --host 127.0.0.1 --port 3001`
  - Observed EPERM bind failure (recorded in Step 5).
- Marked the "Validate" task as complete in ticket tasks, with the explicit note about the dev-server limitation.

Why

- We need confidence that the UI compiles and builds after removing auth, even if we can't fully exercise the dev server in this sandbox.
- Keeping the sqlite files locally is convenient for iterative dev, but they must stay out of git history.

What worked

- `tsc --noEmit` passed.
- `vite build` passed.

What didn't work

- Vite dev server start on `127.0.0.1:3001` failed with EPERM (likely sandbox/environment restriction).

What I learned

- Build/tsc validation gives high confidence the crash is fixed, but it doesn't guarantee runtime routes render; dev-server restrictions can mask that final check.

What was tricky to build

- N/A

What warrants a second pair of eyes

- N/A

What should be done in the future

- Re-validate `pnpm dev:ui` in a normal dev environment and click through the main routes (`/`, `/admin`, `/documents/:slug`) with the Go backend running.

Code review instructions

- Confirm tasks are all checked at `markdown-quizz/ttmp/2026/01/19/MO-001-CLEANUP-LEGACY--cleanup-legacy-version-frontend/tasks.md`.
- Validate locally:
  - `pnpm -C markdown-quizz/legacy-version dev:ui`
  - `pnpm -C markdown-quizz/legacy-version build:ui`

Technical details

- SQLite files are intentionally kept locally and ignored via `markdown-quizz/.gitignore`.